

Backend Developer Assessment – Submission

Muhammed Musthafa S · its.musthafa.s@gmail.com · github.com/its-musthafa

Note: The repo was read-only so I could not push directly. I worked through the solution locally in VSCode using the existing files as reference. Full code is on my GitHub at the link above. This doc explains what I changed and why. Happy to walk through it live in the CTO call.

Problem 1 – API Endpoint Performance

What was wrong

The `getDashboardStats` function ran all `countDocuments` calls one after another sequentially. For the admin role that was 6 database round trips in a row, each waiting for the previous to finish. On top of that there was no caching, so every page load hit the database from scratch.

Fix 1 – Parallel queries

Wrapped all the count calls in `Promise.all` so they run at the same time. Total wait time becomes the slowest single query instead of the sum of all of them.

```
// before
const totalUsers = await User.countDocuments();
const totalPatients = await User.countDocuments({ role: 'patient' });
const totalDoctors = await User.countDocuments({ role: 'doctor' });

// after
const [totalUsers, totalPatients, totalDoctors,
      totalAppointments, totalAnalyses, pendingAppointments]
  = await Promise.all([
    User.countDocuments({}),
    User.countDocuments({ role: 'patient' }),
    User.countDocuments({ role: 'doctor' }),
    Appointment.countDocuments({}),
    AIAnalysis.countDocuments({}),
    Appointment.countDocuments({ status: 'pending' })
  ]);
```

Fix 2 – In-memory cache, 5 minute TTL

Added a simple cache object at the top of the controller. Results are stored per user with an expiry timestamp. If a valid entry exists on the next request it returns immediately without hitting the database.

```
const cache = {
  store: {},
  set(key, value, ttlMs = 5 * 60 * 1000) {
    this.store[key] = { value, expiresAt: Date.now() + ttlMs };
  },
  get(key) {
    const entry = this.store[key];
    if (!entry) return null;
    if (Date.now() > entry.expiresAt) {
      delete this.store[key];
      return null;
    }
    return entry.value;
  },
  invalidate(key) { delete this.store[key]; },
  invalidateAll() { this.store = {}; }
};
```

Fix 3 – Cache cleared on mutations

`deleteUser` calls `cache.invalidateAll()` so counts update after a deletion. Also exported `invalidateStatsCache` so the auth controller can clear it on new user registration.

Fix 4 – Both storage modes

Did not need to touch `modelHelper.js`. The fix works at the query level so MongoDB and in-memory store both benefit through the existing abstraction.

Screenshot – cache and Promise.all in VSCode:

```
7
8  const cache = {
9    store: {},
10
11    set(key, value, ttlMs = 5 * 60 * 1000) {
12      this.store[key] = {
13        value,
14        expiresAt: Date.now() + ttlMs,
15      };
16    },
17
18    get(key) {
19      const entry = this.store[key];
20      if (!entry) return null;
21      if (Date.now() > entry.expiresAt) {
22        delete this.store[key];
23        return null;
24      }
25      return entry.value;
26    },
27
28    invalidate(key) {
29      delete this.store[key];
30    },
31
32    invalidateAll() {
33      this.store = {};
34    },
35  };
36
```

```

if (role === "admin") {
  const [
    totalUsers,
    totalPatients,
    totalDoctors,
    totalAppointments,
    totalAnalyses,
    pendingAppointments,
  ] = await Promise.all([
    User.countDocuments({}),
    User.countDocuments({ role: "patient" }),
    User.countDocuments({ role: "doctor" }),
    Appointment.countDocuments({}),
    AIAnalysis.countDocuments({}),
    Appointment.countDocuments({ status: "pending" }),
  ]);

  stats = {
    totalUsers,
    totalPatients,
    totalDoctors,
    totalAppointments,
    totalAnalyses,
    pendingAppointments,
  };
}

```

Problem 2 – Error Handling and Logging

What was wrong

The original `errorHandler.js` only had `console.error(err)` with nothing else around it. No timestamps, no user context, no request info, no log levels. Sensitive data like passwords could show up in the output. JWT errors were not handled either.

New file: `server/utils/logger.js`

Three methods: `logger.error`, `logger.warn`, `logger.info`. Every entry gets a timestamp and level automatically. In development the output is readable. In production it outputs one JSON object per line, ready for tools like Datadog or CloudWatch. Sensitive fields are replaced with `[REDACTED]` before anything gets written.

```

// dev output
[ERROR] 2026-05-28T10:23:01.000Z - Unhandled server error
{ "userId": "abc123", "path": "/api/users/stats", "method": "GET" }

// production output (one line)
{"timestamp":"2026-05-28T10:23:01.000Z","level":"ERROR",
 "message":"Unhandled server error","userId":"abc123","path":"/api/users/stats"}

// sensitive fields
{ "password": "[REDACTED]", "token": "[REDACTED]" }

```

Updated: `server/middlewares/errorHandler.js`

Swapped `console.error` for the structured logger. Expected errors like validation and bad IDs use `logger.warn`. Actual server crashes use `logger.error` with the stack trace. Added JWT invalid token and JWT expired handling.

```

// context attached to every log entry
const context = {
  userId: req.user?.id || null,
  path: req.path,
  method: req.method
};

```

```
// CastError, duplicate key, validation --> logger.warn
// JWT invalid, JWT expired --> logger.warn
// everything else (500) --> logger.error
```

Logger's Core - Screenshot

```
const logger = {
  /**
   * @param {string} message
   * @param {Error} err
   * @param {object} context - objs
   */
  error(message, err = {}, context = {}) {
    log("error", message, {
      ...context,
      errorMessage: err.message || null,
      stack: err.stack || null,
    });
  },

  // Warninhs about potential issues
  warn(message, context = {}) {
    log("warn", message, context);
  },

  // General info logs
  info(message, context = {}) {
    log("info", message, context);
  },
};

module.exports = logger;
```

Files Changed

- server/controllers/userController.js – added cache, Promise.all, exported invalidateStatsCache
- server/utlis/logger.js – new file, centralized logger
- server/middlewares/errorHandler.js – replaced console.error, added JWT handling

No new packages needed. Everything is built on Node.js built-ins. Cache can be swapped for Redis behind the same interface if needed later. Logger interface works with Winston or Pino as a drop-in.